

République Tunisienne

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Carthage

École Nationale des Sciences et Technologies Avancées

à Borj Cédria

RAPPORT DE STAGE OUVRIER

Élaboré par :

AMAL MECHERGUI

Stage effectué au sein de

MAJESTEYE

du **01/07/2026** au **20/08/2026**

Année universitaire : 2026-2027

Réf. :

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué au bon déroulement et à la réussite de ce stage ouvrier.

J'adresse mes plus sincères remerciements à mon encadrant professionnel au sein de l'entreprise MajestEYE, Monsieur Saad Eddine Khemiri, pour son accueil chaleureux, son encadrement bienveillant, sa disponibilité constante et ses précieux conseils tout au long de cette période. Ses orientations techniques et méthodologiques m'ont permis d'acquérir une expérience pratique inestimable et d'approfondir considérablement mes compétences professionnelles.

Je remercie également la direction et l'ensemble de l'équipe de MajestEYE pour m'avoir offert l'opportunité de réaliser ce stage dans un environnement dynamique et stimulant.

Enfin, j'exprime ma vive reconnaissance à l'ensemble du corps professoral de l'École Nationale des Sciences et Technologies Avancées à Borj Cédria (ENSTAB) pour la qualité de leur formation et leur accompagnement continu.

Table des matières

Remerciements	1
Liste des figures	5
Liste des tableaux	6
Liste des abréviations	7
Introduction Générale	9
1 Contexte général du projet et analyse des besoins	11
1.1 Introduction	11
1.2 Présentation de l'organisme d'accueil	11
1.2.1 Présentation de MajestEYE	11
1.2.2 Activités de l'entreprise	12
1.2.3 Structure et organisation de l'entreprise	12
1.3 Contexte général du projet	13
1.4 Problématique	13
1.5 Étude de l'existant	14
1.5.1 Solutions existantes	14
1.5.2 Limites de l'existant	16
1.6 Solution proposée	17
1.6.1 Application mobile citoyenne	17
1.6.2 Backend de gestion	17
1.6.3 Services d'intelligence artificielle	17
1.6.4 Tableaux de bord analytiques	18
1.7 Spécification des besoins	18
1.7.1 Identification des acteurs du système	18
1.7.2 Besoins fonctionnels	19

1.7.3	Besoins non fonctionnels	20
1.8	Méthodologie de travail et planification	20
1.8.1	Étude comparative entre méthodes agiles et méthodes classiques	20
1.8.2	Le framework Scrum	21
1.8.3	Formalisme adopté	22
1.8.4	Product Backlog (Carnet de produit)	22
1.8.5	Organisation des Sprints	23
1.9	Conclusion	24
2	Architecture globale et conception détaillée par Sprint	25
2.1	Introduction	25
2.2	Architecture générale du système	25
2.3	Modélisation globale du système	26
2.3.1	Diagramme global des cas d'utilisation	26
2.3.2	Diagramme de classes général	27
2.4	Conception détaillée par Sprint	29
2.4.1	Sprint 1 : Conception du Frontend Mobile (Flutter)	29
2.4.2	Sprint 2 : Conception du Backend IA & Moteur conversationnel (Python)	30
2.4.3	Sprint 3 : Conception du Backend de gestion et Sécurité (Spring Boot)	34
2.4.4	Sprint 4 : Conception des Tableaux de bord (Metabase & Portail Technicien)	34
2.4.5	Sprint 5 : Conception de l'Assistant Vocal Multimodal (STT / TTS)	35
2.5	Conclusion	36
3	Réalisation, évaluation et résultats	37
3.1	Introduction	37
3.2	Environnement de développement	37
3.2.1	Environnement matériel	37
3.2.2	Environnement logiciel	38
3.3	Évaluation expérimentale et résultats des modèles IA	40
3.3.1	Classification sémantique des réclamations (NLP)	40
3.3.2	Classification du niveau d'urgence (Random Forest)	41
3.3.3	Évaluation du modèle de vision (Vision Transformer ViT / CLIP)	42
3.4	Présentation des interfaces de la solution	42
3.4.1	Application mobile citoyenne (Flutter)	42
3.4.2	Tableaux de bord opérationnels et décisionnels	45
3.5	Apport de l'application	46
3.6	Limites de l'application	46

3.7 Conclusion	47
Conclusion Générale	48
Références bibliographiques	49

Table des figures

1.1	Logo de MajestEYE	12
1.2	Organigramme de MajestEYE	12
1.3	Interface de réclamation de la SONEDE	14
1.4	Interface de la plateforme e-People Tunisia	15
1.5	Interface de gestion des demandes de Zoho Desk	15
1.6	Méthode Agile VS Méthode Classique	21
1.7	Cycle de vie de la méthode Scrum	21
2.1	Architecture générale de la plateforme	26
2.2	Diagramme global des cas d'utilisation	27
2.3	Diagramme de classes général du système	28
2.4	Diagramme de navigation de l'application mobile citoyenne	29
2.5	Architecture globale du pipeline RAG intégré au chatbot citoyen	31
2.6	Pipeline de traitement du langage naturel	32
2.7	Pipeline d'analyse des images	33
2.8	Diagramme de séquence de l'authentification sécurisée par JWT	34
2.9	Pipeline de traitement de l'assistant vocal multimodal	36
3.1	Logo Flutter	38
3.2	Logo Spring Boot	38
3.3	Logo Flask	38
3.4	Logo MongoDB	39
3.5	Logo Hugging Face	39
3.6	Interface web de la plateforme Metabase pour la création de tableaux de bord	39
3.7	Logo VS Code	40
3.8	Matrice de confusion - Classification des catégories d'intention	41
3.9	Matrice de confusion - Détection du niveau d'urgence	41
3.10	Interface de connexion de l'application citoyenne	43
3.11	Interface du chatbot conversationnel multimodal	43

3.12 Interface de l'assistant vocal interactif	44
3.13 Interface de suivi en temps réel des réclamations	44
3.14 Interface de gestion du profil citoyen	45
3.15 Tableau de bord destiné aux techniciens (Suivi des interventions)	45
3.16 Tableau de bord destiné aux administrateurs sous Metabase	46

Liste des tableaux

1.1	Comparaison des solutions existantes et de la solution proposée	17
1.2	Product Backlog général du projet	23
1.3	Planification chronologique des sprints	23
3.1	Caractéristiques de l'environnement matériel	37
3.2	Rapport de classification du modèle de détection des catégories	40
3.3	Résultats des tests de qualification d'images avec le Vision Transformer (ViT) .	42

Liste des abréviations

API	Application Programming Interface
ASR	Automatic Speech Recognition
BI	Business Intelligence
CLIP	Contrastive Language-Image Pre-Training
CPU	Central Processing Unit
GPU	Graphics Processing Unit
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
IA / AI	Intelligence Artificielle / Artificial Intelligence
JWT	JSON Web Token
KPI	Key Performance Indicator (Indicateur Clé de Performance)
LLM	Large Language Model
MTTR	Mean Time to Repair (Temps Moyen de Réparation)
NER	Named Entity Recognition (Reconnaissance d'Entités Nommées)
NLP / TALN	Natural Language Processing / Traitement Automatique du Langage Naturel
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer
SONEDE	Société Nationale d'Exploitation et de Distribution des Eaux
SQL	Structured Query Language
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface
UML	Unified Modeling Language

US	User Story
UX	User Experience
ViT	Vision Transformer

Introduction Générale

Le stage ouvrier constitue une étape fondamentale dans la formation de l'élève ingénieur. Il offre une première immersion au sein du monde professionnel, permettant de découvrir le fonctionnement d'une entreprise, d'observer l'organisation du travail en équipe et d'appréhender des problématiques industrielles et citoyennes réelles.

Dans le cadre de la transition numérique des services publics en Tunisie, la gestion efficace des services d'eau potable représente un enjeu stratégique. La Société Nationale d'Exploitation et de Distribution des Eaux (SONEDE) assure l'acheminement de l'eau sur tout le territoire national. Toutefois, le traitement des réclamations citoyennes, qu'il s'agisse de fuites sur la voie publique, de coupures non planifiées ou d'anomalies de compteurs, repose encore largement sur des canaux conventionnels lents et manuels, souvent peu adaptés aux habitudes d'expression des usagers, notamment en dialecte tunisien.

C'est dans ce cadre que s'inscrit ce stage ouvrier réalisé au sein de l'entreprise MajestEYE. L'objectif a été de concevoir et de développer une solution innovante que la SONEDE peut intégrer pour optimiser ses processus de gestion des réclamations, accélérer le traitement des pannes et fluidifier la communication avec les usagers. La plateforme intègre une application mobile sous Flutter dotée d'un chatbot et d'un assistant vocal, un backend sécurisé sous Spring Boot, des microservices d'intelligence artificielle en Python combinant le traitement automatique du langage naturel en dialecte tunisien, une architecture RAG et un modèle Vision Transformer pour l'analyse d'images, ainsi que des tableaux de bord analytiques sous Metabase.

Ce rapport relate le déroulement de cette expérience professionnelle et s'organise en trois chapitres.

Le premier chapitre présente l'environnement du stage chez MajestEYE, l'analyse de la problématique dans le contexte de la SONEDE, l'étude de l'existant, la spécification des besoins ainsi que la méthodologie agile Scrum adoptée.

Le deuxième chapitre expose la conception architecturale globale du système et détaille la modélisation logique des différents sprints de développement.

Enfin, le troisième chapitre décrit l'environnement matériel et logiciel de développement, les résultats expérimentaux des modèles d'intelligence artificielle, les interfaces finales réalisées, ainsi que le bilan des tests d'intégration.

Chapitre 1

Contexte général du projet et analyse des besoins

1.1 Introduction

Avec la transformation numérique qui devient une réalité il y a des opportunités qui sont apparues pour améliorer la gestion des plaintes des citoyens et l'optimisation de l'utilisation des ressources. Dans un tel scénario le projet actuel mis en œuvre dans le cadre d'un stage chez MajestEYE vise la conception d'une solution qui répondra aux besoins de SONEDE. Ce chapitre présente les détails de la société hôte, la situation générale du projet, le problème et la solution proposée.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation de MajestEYE

Fondée en 2017 par M. Jihed Hannachi, **MajestEYE** est une entreprise spécialisée dans les domaines de l'analyse de données et de l'Intelligence Artificielle. La société assiste ses clients dans la valorisation de leurs données et dans le développement d'une solution analytique selon leurs besoins.

La société opère dans différents secteurs tels que la finance, la santé, le secteur public et l'industrie manufacturière. Les compétences qui sont au cœur de cette entreprise sont la complémentarité des compétences en science des données, en ingénierie des données, en technologie et en Intelligence Artificielle.



FIGURE 1.1 – Logo de MajestEYE

1.2.2 Activités de l'entreprise

Les opérations de MajestEYE consistent principalement à collecter, traiter et analyser les données afin de résoudre les problèmes auxquels sont confrontés ses clients. MajestEYE réunit des domaines de connaissances tels que l'ingénierie des données, l'intelligence artificielle, l'ingénierie logicielle et l'analyse avancée. Ainsi MajestEYE fournit ses services pour les besoins spécifiques des entreprises et la mise en œuvre de technologies intelligentes dans leurs processus de travail.

1.2.3 Structure et organisation de l'entreprise

La structure de MajestEYE est caractérisée par quelques piliers qui se complètent et qui concernent principalement le développement de la technologie, des opérations, la recherche et le développement ainsi que les activités liées à l'intelligence artificielle et aux données.

L'entreprise emploie des profils ayant des niveaux de compétences variés, notamment dans les domaines de l'ingénierie des données, de la science des données, de l'intelligence artificielle et du développement logiciel. La structure de l'entreprise encourage la collaboration entre les différents départements et permet l'accès à l'expertise requise afin de développer des solutions technologiques adaptées à leurs clients.

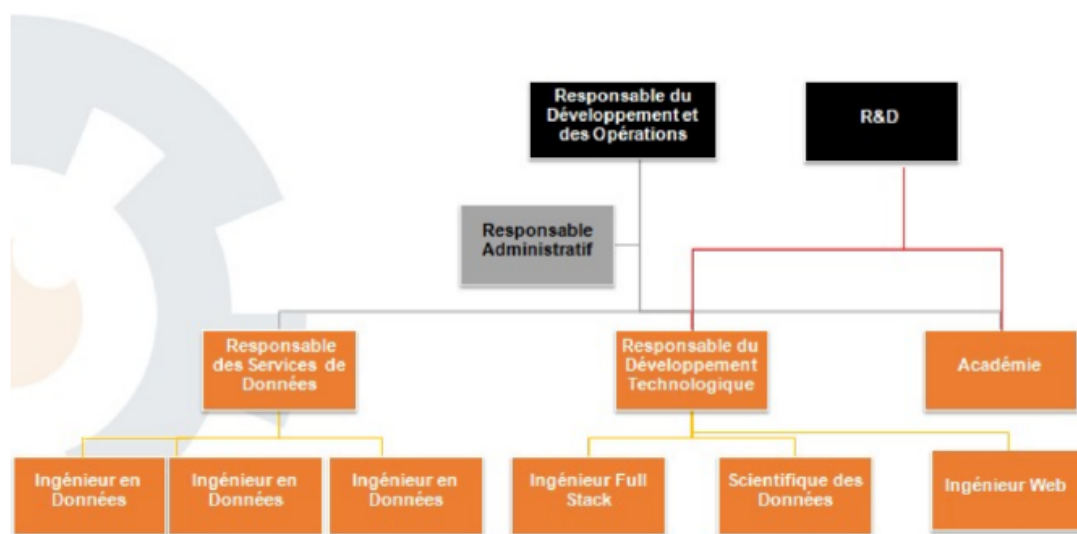


FIGURE 1.2 – Organigramme de MajestEYE

1.3 Contexte général du projet

Dans le cadre de ce stage réalisé au sein de MajestEYE, le projet porte sur la conception et le développement d'une solution intelligente destinée à améliorer la gestion des réclamations citoyennes dans le contexte des services de distribution de l'eau.

La Société Nationale d'Exploitation et de Distribution des Eaux (SONEDE) est l'établissement public tunisien chargé de la production, de l'acheminement et de la distribution de l'eau potable sur l'ensemble du territoire national. Dans le cadre de ce projet, son environnement opérationnel constitue le contexte d'application étudié, notamment en ce qui concerne la réception, la qualification et le suivi des réclamations liées aux incidents affectant les services de distribution de l'eau.

Les signalements peuvent concerner différentes situations, telles que les fuites d'eau sur la voie publique, les ruptures de canalisations, Les coupures ou certaines anomalies liées aux compteurs. La diversité de ces demandes et les informations qu'elles peuvent contenir rendent nécessaire l'utilisation d'outils permettant de faciliter leur collecte, leur compréhension et leur traitement.

1.4 Problématique

La gestion des réclamations peut être confrontée à plusieurs contraintes susceptibles de ralentir leur traitement et de rendre leur orientation plus complexe. Le volume important des demandes peut notamment compliquer leur réception, leur qualification et leur suivi lorsqu'ils reposent principalement sur des moyens traditionnels.

La diversité des modes d'expression est un autre grand défi. En effet les citoyens peuvent exprimer leurs demandes en français ,en arabe ,en dialecte tunisien ou en Arabizi. Cela peut compliquer la compréhension et le traitement de leur demande. De plus certaines réclamations peuvent manquer de détails suffisants sur la situation ou l'incident décrit. L'option d'inclure différents types de contenus tels que des photos ou des messages vocaux sera donc nécessaire afin de faciliter leur description. Il existe des différences de besoins selon le profil impliqué dans le traitement des réclamations. En effet les techniciens ont besoin d'informations précises pour le traitement d'un incident et le suivi d'une intervention tandis que les managers ont besoin d'une vue d'ensemble de l'activité et d'indicateurs facilitant le suivi.

Tous ces défis mettent en évidence le besoin d'un système centralisé et intelligent qui puisse aider à la collecte ,au traitement ,à l'analyse et au routage des réclamations ainsi qu'au suivi de leurs activités.

1.5 Étude de l'existant

Avant de présenter la solution proposée, il est utile d'examiner quelques solutions déjà utilisées pour la gestion des réclamations citoyennes. Cette étude permet d'identifier leurs principales fonctionnalités et de relever les besoins qui restent à couvrir.

1.5.1 Solutions existantes

Dispositif actuel de la SONEDE

La SONEDE propose plusieurs moyens pour déposer une réclamation. Vous pouvez déposer une réclamation via le site web de la SONEDE ou appeler le numéro vert de la SONEDE. La SONEDE transmet ensuite la demande à son district concerné et assure le suivi de chaque réclamation jusqu'à son traitement. Ce dispositif s'intègre directement à l'organisation de la SONEDE et garantit un suivi fiable des réclamations. Cependant, la SONEDE s'appuie surtout sur des moyens classiques de communication et ne fournit pas d'outils d'intelligence artificielle pour analyser automatiquement les réclamations.[1]

La figure 1.3 présente l'interface utilisée par la SONEDE pour la gestion des réclamations.



FIGURE 1.3 – Interface de réclamation de la SONEDE

e-People Tunisia

e-People Tunisie permet aux citoyens de présenter leurs réclamations à l'administration et de consulter en ligne l'évolution de leurs demandes. Cela simplifie certaines démarches et permet d'éviter de tout faire en personne sur place. Cependant, la plateforme reste assez générale. Elle ne gère pas vraiment les problèmes concernant le réseau de distribution d'eau. Notre projet vise précisément à résoudre ce problème en proposant des fonctionnalités intelligentes et diverses méthodes pour signaler une issue. [2]

La figure 1.4 présente l'interface de la plateforme e-People Tunisia destinée aux démarches des citoyens.



FIGURE 1.4 – Interface de la plateforme e-People Tunisia

Zoho Desk

Zoho Desk est une solution qui permet de gérer les demandes de manière efficace. centraliser les demandes, suivre leur traitement et les assigner. aux agents concernés. Elle propose également un portail multilingue et différents outils d'automatisation. Cette solution propose plusieurs fonctionnalités pratiques pour gérer. des réclamations. Elle reste cependant générale et doit être ajustée. aux besoins spécifiques de chaque domaine.[3]

La figure 1.5 présente un aperçu de l'interface de gestion des demandes de Zoho Desk.

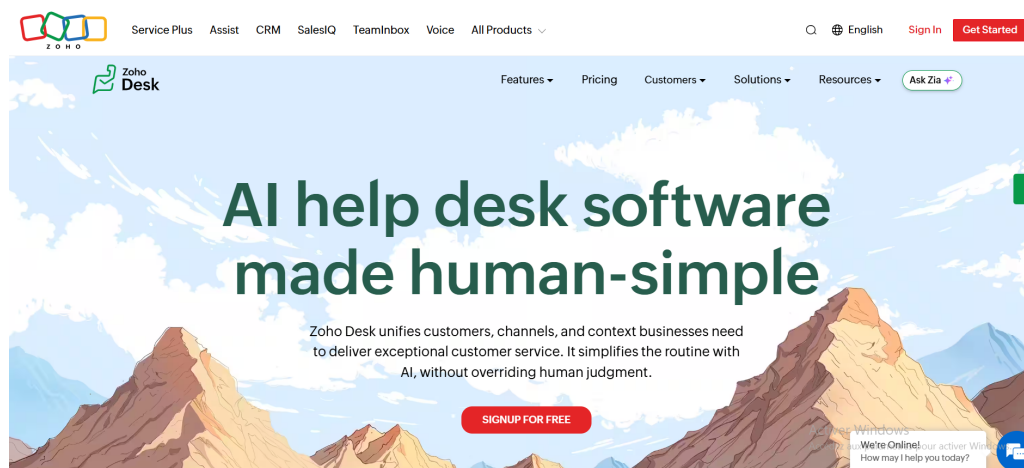


FIGURE 1.5 – Interface de gestion des demandes de Zoho Desk

Smart Civic AI

Smart Civic AI est une solution basée sur l'intelligence artificielle pour la gestion des réclamations citoyennes. Elle permet notamment d'utiliser des images et la voix pour signaler un problème. Elle intègre également la localisation GPS, la gestion des priorités, la visualisation par carte et des tableaux de bord adaptés aux différents acteurs.[4]

1.5.2 Limites de l'existant

L'étude des canaux actuels de gestion des réclamations (guichets physiques, standards téléphoniques et formulaires en ligne classiques) met en évidence plusieurs insuffisances majeures :

Canaux de communication rigides et peu inclusifs La majorité des solutions existantes imposent la saisie de formulaires textuels en français ou en arabe standard. Ces interfaces excluent les usagers ayant des difficultés avec l'écrit ou peu à l'aise avec les outils numériques. De plus, l'absence de prise en compte du dialecte tunisien (Derja) et de l'Arabizi freine l'expression naturelle des citoyens lors du signalement d'un incident.

Traitement manuel et saturation des services La qualification des réclamations repose encore sur la lecture manuelle de chaque message par des agents administratifs. Ce processus entraîne des délais de traitement importants, des erreurs de classement et des difficultés à repérer les signalements multiples (doublons) concernant une même fuite d'eau sur le réseau.

Manque de visibilité analytique pour les décideurs Les outils actuels se limitent souvent à des fichiers de suivi statiques ou à des bases de données isolées. Les responsables manquent d'indicateurs visuels et géographiques en temps réel pour cartographier les pannes récurrentes, mesurer les temps d'intervention et organiser une maintenance préventive efficace.

Rupture de communication avec le terrain et l'utilisateur La transmission des réclamations vers les techniciens sur le terrain reste lente et peu structurée. En l'absence d'outils mobiles interconnectés, les techniciens peinent à recevoir rapidement les détails des pannes. De son côté, le citoyen reste sans retour sur l'avancement de sa demande, ce qui génère de la frustration et une perte de confiance envers le service public. Le tableau 1.1 illustre bien la différence entre les solutions existantes et la solution proposée

TABLE 1.1 – Comparaison des solutions existantes et de la solution proposée

Solution	Multimodal	IA	Assistant vocal	Dashboard	Adaptation SONED
Dispositif SONED	Partiel	Faible	Non	Limité	Forte
e-People Tunisia	Limité	Faible	Non	Oui	Faible
Zoho Desk	Oui	Oui	Non	Oui	Moyenne
Smart Civic AI	Oui	Oui	Oui	Oui	Faible
Solution proposée	Oui	Oui	Oui	Oui	Forte

1.6 Solution proposée

Pour moderniser la gestion des réclamations, nous avons conçu une plateforme complète qui relie directement les usagers aux équipes techniques. Le système prend en charge chaque étape : la déclaration par le citoyen, l'analyse automatique par l'intelligence artificielle, l'intervention sur le terrain et le suivi des statistiques par la direction. L'architecture repose sur quatre briques principales.

1.6.1 Application mobile citoyenne

L'application mobile est développée avec Flutter. Elle sert de point d'entrée pour les usagers. L'accent a été mis sur la simplicité pour permettre à tout citoyen de signaler un problème rapidement. L'utilisateur dispose de plusieurs moyens pour décrire un incident : saisir un texte, joindre des photos ou enregistrer un message vocal en dialecte tunisien (Derja). Un chatbot et un assistant vocal guident les personnes qui ont des difficultés avec la saisie manuelle. L'application permet aussi de consulter l'historique et l'état d'avancement des demandes déposées.

1.6.2 Backend de gestion

Le backend centralise les données et gère la logique métier de la plateforme. Il assure la communication entre l'application mobile, les services d'IA et la base de données. Ce module prend en charge l'authentification et le contrôle d'accès par rôles (RBAC). Les données restent sécurisées et chaque utilisateur (citoyen, technicien ou administrateur) accède uniquement aux fonctions qui lui sont attribuées.

1.6.3 Services d'intelligence artificielle

Les services d'IA sont développés en Python. Leur but est d'automatiser le tri et la qualification des réclamations dès leur réception. Ces modules combinent le traitement du langage naturel,

l'analyse d'images et la reconnaissance vocale. Le système extrait le type de panne et le lieu à partir des messages ou des audios, détecte les anomalies visibles sur les photos et repère les doublons. La réclamation est alors classée et transmise au bon service sans intervention manuelle préalable.

1.6.4 Tableaux de bord analytiques

La plateforme propose deux interfaces de suivi adaptées aux besoins de chaque équipe :

Le portail technicien est une interface web opérationnelle. Les techniciens y consultent les réclamations qui leur sont assignées, visualisent les informations extraites par l'IA et mettent à jour le statut des interventions après leur passage.

Le portail administrateur intègre des tableaux de bord interactifs réalisés avec Metabase. Cet espace permet aux responsables de suivre les indicateurs globaux : volume des pannes par zone, temps moyen de réparation et motifs fréquents de réclamation, afin de mieux planifier la maintenance du réseau.

1.7 Spécification des besoins

La conception de la plateforme repose sur une analyse approfondie des attentes des usagers du service public d'eau ainsi que des contraintes opérationnelles des équipes de maintenance et de gestion. Cette section détaille les acteurs cibles, les exigences fonctionnelles réparties par rôle, ainsi que les critères de qualité non fonctionnels garantissant la viabilité du système.

1.7.1 Identification des acteurs du système

Le système fait intervenir trois catégories d'utilisateurs aux prérogatives distinctes :

- Le Citoyen : Il représente l'utilisateur final du service. Il signale des anomalies constatées sur le réseau (fuites d'eau, coupures d'alimentation, problèmes de compteurs ou litiges de facturation), interagit en langage naturel (français ou dialecte tunisien) avec les agents conversationnels et suit l'évolution de ses requêtes.
- Le Technicien / Agent de maintenance : Acteur de terrain et de régulation, il accède via son portail dédié aux réclamations qui lui sont affectées, consulte les pré-analyses générées par l'IA (diagnostic visuel, degré de priorité, localisation extraite) et rend compte de l'avancement des réparations.
- L'Administrateur / Décideur : Responsable du pilotage global, il gère les habilitations des utilisateurs, supervise le flux global des incidents et exploite les tableaux de bord analytiques pour orienter la prise de décision stratégique.

1.7.2 Besoins fonctionnels

Les fonctionnalités attendues sont structurées selon les parcours d'usage de chaque intervenant.

Besoins du citoyen (Application mobile)

L'application mobile met l'accent sur l'inclusivité et la simplicité de déclaration. Le citoyen doit pouvoir :

- Créer un compte sécurisé, s'authentifier et gérer son profil personnel.
- Soumettre un signalement par saisie textuelle ou enregistrement vocal, avec prise en charge du dialecte tunisien (Derja).
- Joindre des photographies de l'incident pour une analyse visuelle automatique.
- Dialoguer avec un chatbot interactif pour être guidé étape par étape dans la description de sa panne.
- Utiliser un assistant vocal complet (reconnaissance de la parole et synthèse vocale) pour formuler des requêtes sans saisie manuelle.
- Suivre en direct le changement de statut de ses réclamations (reçue, en cours de traitement, résolue).

Besoins du technicien (Portail opérationnel)

Le portail web technicien vise à optimiser les interventions sur site. L'agent doit pouvoir :

- Visualiser l'ensemble des réclamations assignées selon leur niveau d'urgence et leur localisation géographique.
- Consulter les pré-diagnostic automatisés fournis par les services d'IA (catégorie détectée, transcription audio, détection visuelle de fuite et score de confiance).
- Identifier les réclamations redondantes grâce au mécanisme de détection automatique des doublons sur une même zone.
- Consulter les pièces jointes (photos annotées, pistes audio originales).
- Mettre à jour l'état d'avancement des interventions (prise en charge, intervention planifiée, clôture avec observations).

Besoins de l'administrateur (Gestion & Pilotage BI)

L'espace administratif centralise le contrôle du système et l'analyse décisionnelle. L'administrateur doit pouvoir :

- Administrer les comptes utilisateurs et attribuer les rôles et droits d'accès (RBAC).

- Suivre les indicateurs clés de performance (volume de réclamations par type, délais moyens de résolution, répartition géographique des pannes).
- Consulter et filtrer les données consolidées via les tableaux de bord décisionnels Metabase.
- Analyser les tendances historiques afin d'anticiper les besoins de maintenance préventive sur le réseau.

1.7.3 Besoins non fonctionnels

Pour garantir la pérennité, la robustesse et l'adoption de la solution, plusieurs exigences techniques et ergonomiques sont requises :

- Sécurité et intégrité des données : Authentification basée sur des jetons sécurisés (JWT), contrôle rigoureux des accès aux API et protection des données personnelles des citoyens.
- Ergonomie et accessibilité : Interface mobile intuitive, adaptée aux utilisateurs peu familiers du numérique grâce à l'assistance vocale et à la gestion du langage familier/dialectal.
- Temps de réponse et réactivité : Traitement rapide des requêtes de classification et des transcriptions audio par les microservices d'IA pour ne pas bloquer l'expérience utilisateur.
- Modularité architecturale : Séparation claire entre l'application mobile (Flutter), le backend de gestion (Spring Boot), les services d'intelligence artificielle (Python/Flask) et la Business Intelligence (Metabase), facilitant les évolutions futures.
- Tolérance aux pannes et scalabilité : Capacité à absorber des pics d'affluence lors d'incidents majeurs sur le réseau sans dégradation critique des services essentiels.

1.8 Méthodologie de travail et planification

section Choix méthodologique et formalisme adopté

Le développement logiciel nécessite une méthodologie bien définie. Nous avons choisi d'utiliser Scrum, un framework agile fondé sur l'itération, la collaboration et l'adaptabilité.

1.8.1 Étude comparative entre méthodes agiles et méthodes classiques

Les méthodes classiques (comme le cycle en V ou le modèle en cascade) se caractérisent par une approche rigide et séquentielle. En revanche, les méthodes agiles privilégient l'interaction avec les parties prenantes, les itérations fréquentes, et l'adaptabilité face aux changements.[5]

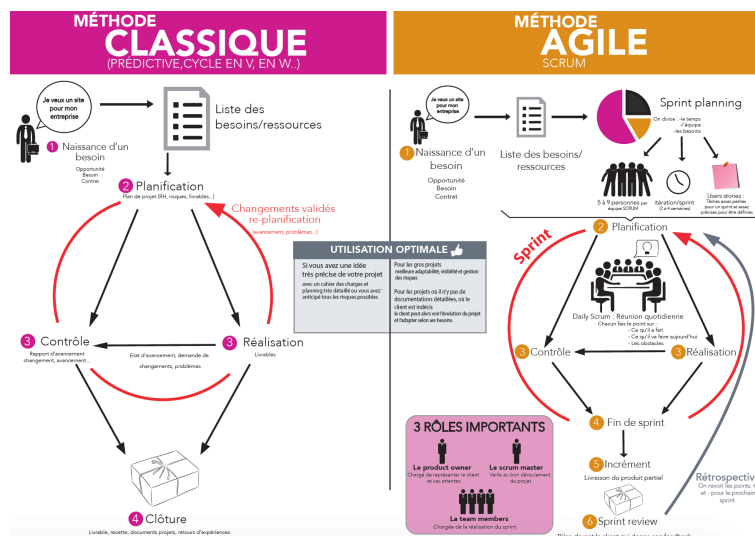


FIGURE 1.6 – Méthode Agile VS Méthode Classique

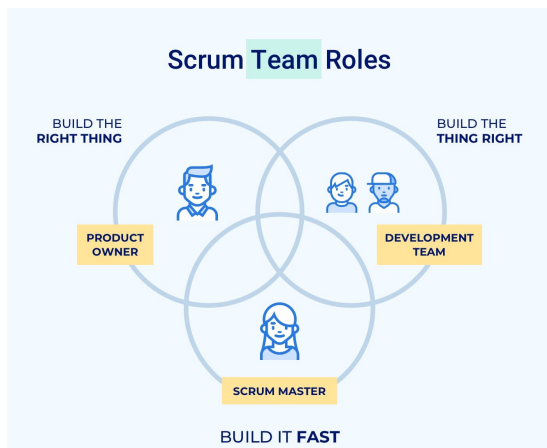
1.8.2 Le framework Scrum

Scrum est un cadre agile permettant de gérer efficacement le développement logiciel. Il repose sur des cycles courts appelés sprints, des réunions régulières (daily meetings), des livraisons fréquentes et une amélioration continue basée sur le retour d'expérience. [6]



FIGURE 1.7 – Cycle de vie de la méthode Scrum

Les rôles dans Scrum :



Le Product Owner (Propriétaire du produit) :

« Responsable de maximiser la valeur du produit et du travail de l'Équipe de Développement. Il gère le carnet de produit (Product Backlog). » [7]

Le Scrum Master : « Responsable de l'application de Scrum et de la conformité de l'Équipe Scrum aux pratiques et règles de Scrum. » [7]

L'équipe de développement (Development Team) : « Professionnels livrant des incréments terminés et potentiellement livrables à chaque Sprint. L'équipe est autonome dans l'organisation de son travail. » [7]

1.8.3 Formalisme adopté

Après avoir sélectionné la méthode, nous devons utiliser un langage de modélisation unifié pour modéliser le projet. Lors de la conception du système, nous avons choisi UML comme langage de modélisation. Ceci est basé sur les avantages du langage, en particulier sa standardisation et les différents diagrammes qu'il fournit, ainsi que les meilleurs outils proposés par UML, qui peuvent simplifier et standardiser des systèmes complexes.

1.8.4 Product Backlog (Carnet de produit)

Le Product Backlog comme représenté dans le tableau 1.2 regroupe les exigences sous forme de *User Stories* priorisées selon la méthode MoSCoW :

TABLE 1.2 – Product Backlog général du projet

ID	Module	User Story	Priorité
US01	Authentification	En tant qu'utilisateur, je veux créer un compte et m'authentifier avec JWT pour accéder à mes services.	Must
US02	Authentification	En tant qu'administrateur, je veux gérer les rôles (RBAC) pour restreindre l'accès aux données.	Must
US03	Application Mobile	En tant que citoyen, je veux soumettre une réclamation (texte et photo) pour signaler une panne.	Must
US04	Application Mobile	En tant que citoyen, je veux suivre le statut de mes réclamations en temps réel.	Must
US05	Intelligence Artificielle	En tant que système, je veux classer automatiquement les pannes et extraire la localisation.	Must
US06	Intelligence Artificielle	En tant que système, je veux comprendre le dialecte tunisien (Derja) pour faciliter l'accès.	Must
US07	Chatbot	En tant que citoyen, je veux dialoguer avec un chatbot interactif pour décrire ma panne étape par étape.	Should
US08	Intelligence Artificielle	En tant que système, je veux détecter les doublons de réclamations sur une même zone géographique.	Should
US09	Tableau de bord Admin	En tant qu'administrateur, je veux visualiser les KPIs et la cartographie des pannes via Metabase.	Should
US10	Portail Technicien	En tant que technicien, je veux consulter mes tâches assignées et clôturer les interventions résolues.	Must
US11	Assistant Vocal	En tant que citoyen, je veux dicter ma réclamation et écouter la réponse vocale (STT/TTS).	Should

1.8.5 Organisation des Sprints

Le tableau 1.3 présente le plan de déroulement des sprints, reflétant l'ordre de conception et d'intégration des modules :

TABLE 1.3 – Planification chronologique des sprints

Sprint	Principales réalisations	Priorité
Sprint 0	Cadrage, analyse du contexte SONED, spécifications et mise en place des environnements.	Haute
Sprint 1	Frontend mobile (Flutter) : Interfaces citoyennes, formulaires texte/photo, navigation.	Haute
Sprint 2	Backend IA & Chatbot : Services NLP Python, prise en compte de la Derja, classification.	Haute
Sprint 3	Backend de gestion : API REST Spring Boot, authentification JWT, rôles RBAC, persistance.	Haute
Sprint 4	Tableaux de bord : Dashboard Admin Metabase (KPIs, cartes) et Portail Web Technicien.	Moyenne
Sprint 5	Assistant vocal & Clôture : Reconnaissance STT et synthèse TTS, intégration globale et tests.	Moyenne

1.9 Conclusion

Ce premier chapitre a permis de définir le cadre de l'étude, d'identifier les besoins des intervenants et d'arrêter la démarche méthodologique Scrum. Le chapitre suivant sera consacré à la conception détaillée du système, en présentant l'architecture globale ainsi que la conception spécifique de chaque sprint.

Chapitre 2

Architecture globale et conception détaillée par Sprint

2.1 Introduction

Après avoir établi les exigences fonctionnelles et la planification méthodologique, cette étape aborde la phase de conception du système. L'objectif est de définir une architecture robuste, évolutive et sécurisée, répondant aux contraintes d'interopérabilité entre les technologies mobiles, les microservices d'IA et les outils décisionnels. Ce chapitre débute par la présentation de l'architecture globale du système et des diagrammes UML généraux (cas d'utilisation et diagramme de classes). Il détaille ensuite la conception logique et technique de chaque sprint, en exposant les flux de données, les algorithmes retenus et les choix de modélisation associés.

2.2 Architecture générale du système

La plateforme adopte une architecture modulaire et distribuée organisée en quatre couches communicant via des API REST standardisées. La couche Présentation regroupe l'application mobile Flutter multiplateforme destinée aux citoyens ainsi que le portail web technicien dédié à la gestion des interventions sur le terrain. La couche Métier et Sécurité, basée sur Spring Boot, centralise la logique métier, la gestion des utilisateurs, l'orchestration des flux de réclamations, ainsi que la sécurité reposant sur des jetons JWT et un contrôle d'accès basé sur les rôles (RBAC). La couche Intelligence Artificielle, implémentée sous forme de microservices Python/Flask, regroupe les modules spécialisés dans le traitement automatique du langage naturel (NLP), avec prise en charge du dialecte tunisien, la vision par ordinateur pour la qualification des images, la gestion du dialogue conversationnel et la reconnaissance vocale. Enfin, la couche Données et

Décisionnel repose sur MongoDB pour le stockage persistant des données et sur Metabase pour leur agrégation, leur analyse et la visualisation des indicateurs stratégiques.

La figure 2.1 ci-dessous présente l'architecture générale adoptée pour la plateforme.

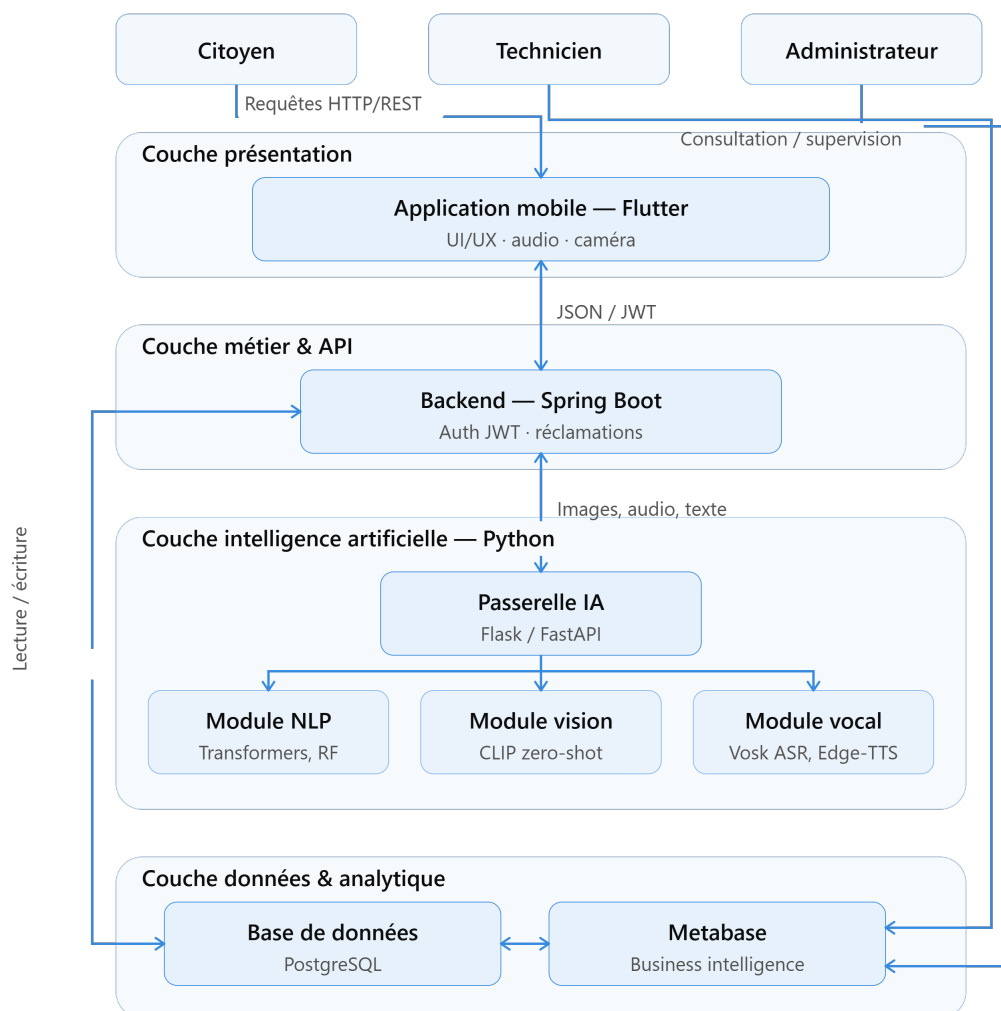


FIGURE 2.1 – Architecture générale de la plateforme

2.3 Modélisation globale du système

2.3.1 Diagramme global des cas d'utilisation

Le diagramme de la figure 2.2 synthétise l'ensemble des interactions entre les trois acteurs principaux et la plateforme :

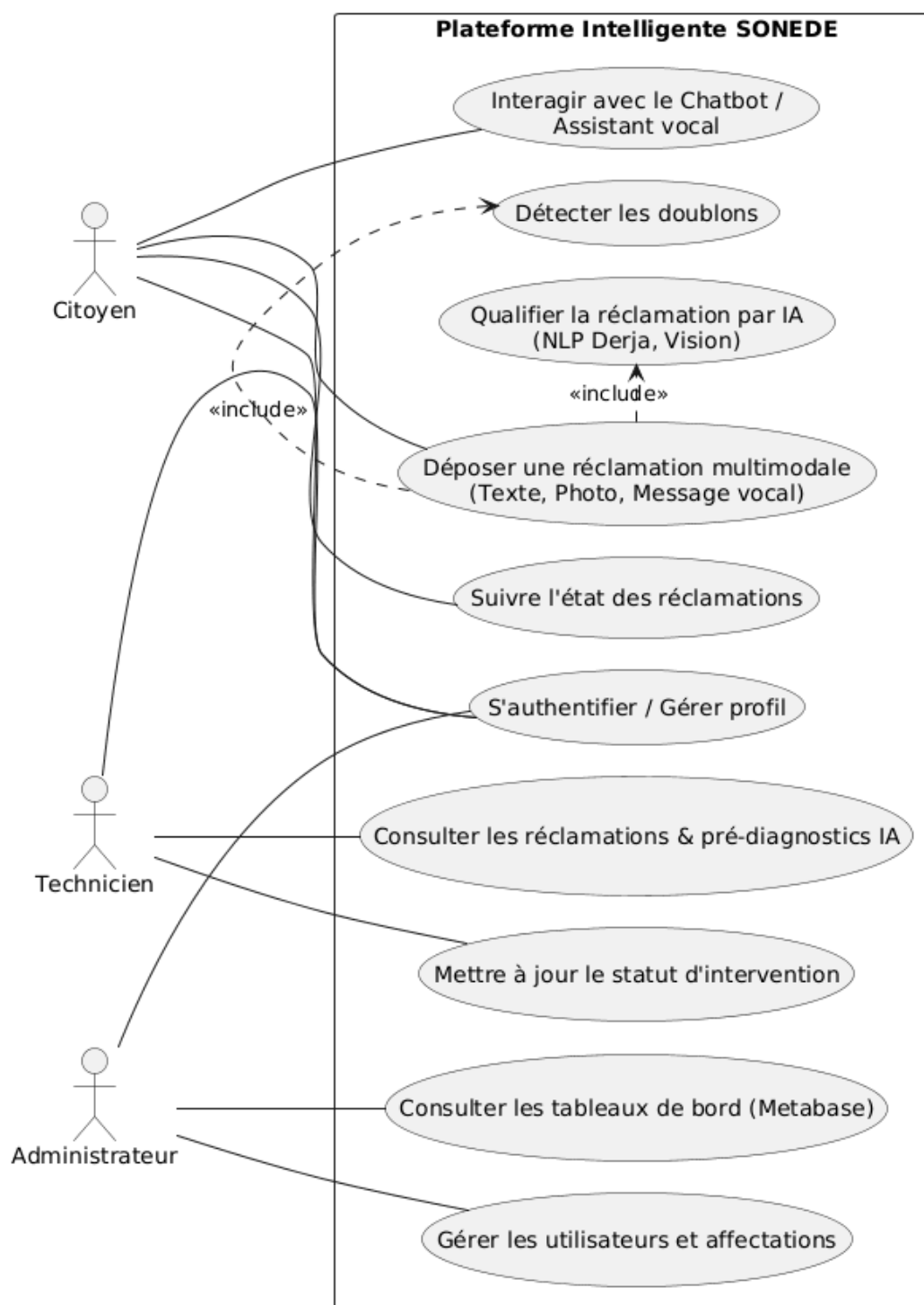


FIGURE 2.2 – Diagramme global des cas d'utilisation

2.3.2 Diagramme de classes général

La structure conceptuelle des données et les relations métier sont illustrées par le diagramme de classes de la figure 2.3 :

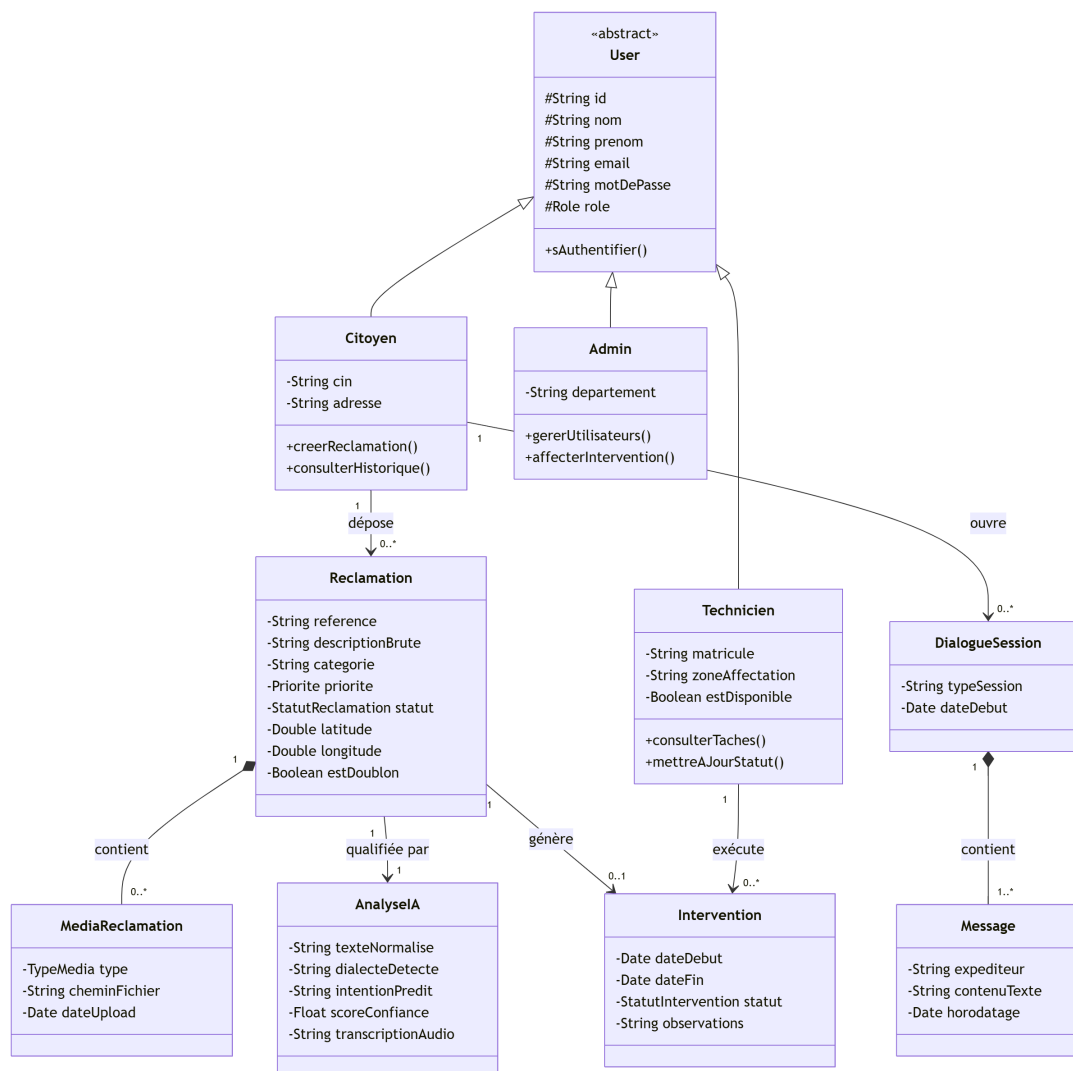


FIGURE 2.3 – Diagramme de classes général du système

Les entités principales comprennent :

- User et ses spécialisations (Citoyen, Technicien, Admin) définissant les identités et permissions.
- Reclamation encapsulant les détails du signalement, sa localisation, son statut et sa priorité calculée par l'IA.
- MediaReclamation stockant les pièces jointes associées (images, enregistrements audio).
- Intervention et DialogueSession assurant la traçabilité des opérations de maintenance et des échanges conversationnels.

2.4 Conception détaillée par Sprint

2.4.1 Sprint 1 : Conception du Frontend Mobile (Flutter)

Ce premier sprint est dédié à la conception de l'application mobile citoyenne avec le framework Flutter. L'architecture utilise le patron *Provider* pour gérer l'état de l'application de façon réactive et séparer l'interface utilisateur des appels aux API REST.

Comme l'illustre le diagramme de navigation de la figure 2.4, l'application vérifie la présence d'un jeton d'authentification dès le lancement pour rediriger l'utilisateur vers la connexion ou directement vers l'accueil. Une barre de navigation inférieure permet ensuite d'accéder aux différentes fonctionnalités : le dépôt d'une réclamation (texte et photos), le suivi en temps réel de l'état des demandes (*En attente*, *En cours*, *Résolue*), les agents conversationnels et la gestion du profil.

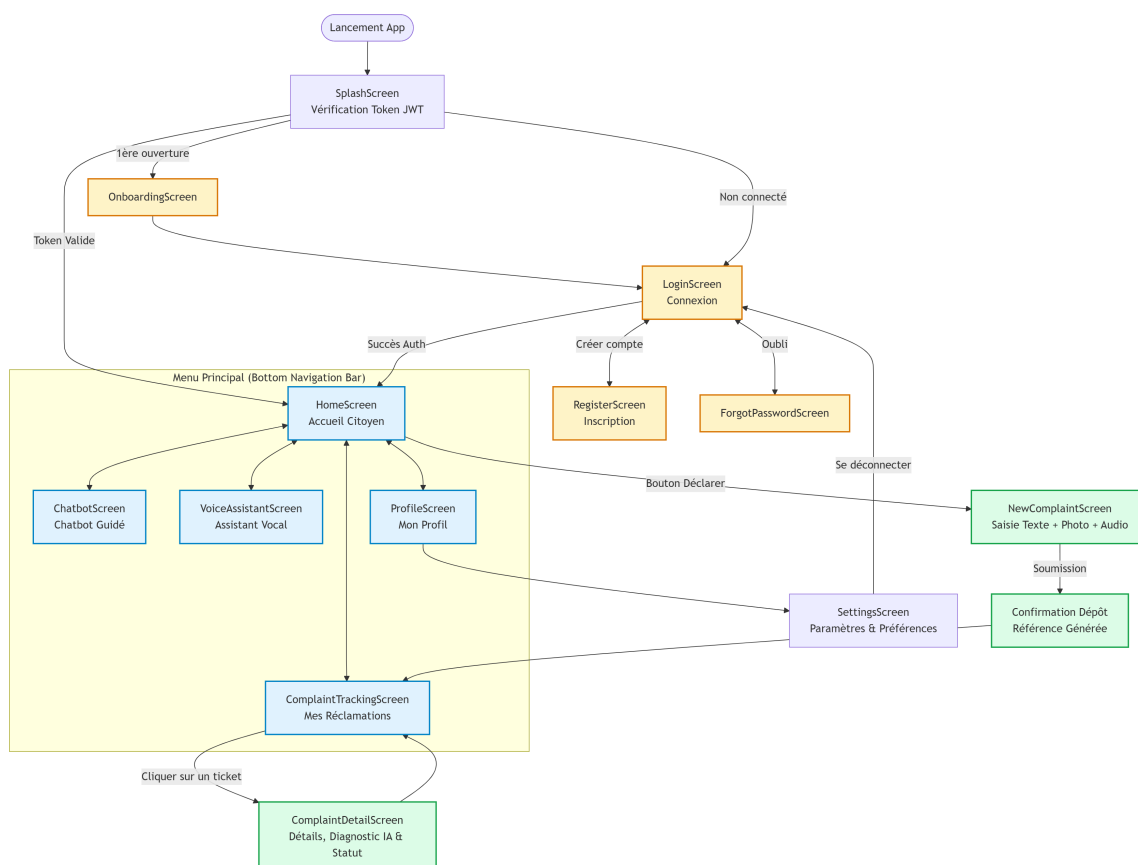


FIGURE 2.4 – Diagramme de navigation de l'application mobile citoyenne

2.4.2 Sprint 2 : Conception du Backend IA & Moteur conversationnel (Python)

Le deuxième sprint constitue le cœur décisionnel de la plateforme. Développé en Python, ce module assure la compréhension sémantique des réclamations, l'extraction des données clés, la qualification visuelle des pannes et le pilotage du moteur conversationnel. Le principal défi réside dans la gestion de la diversité linguistique, les usagers alternant fréquemment entre le français, l'arabe et le dialecte tunisien (Derja en caractères arabes ou en Arabizi). L'architecture intègre ainsi une chaîne RAG (*Retrieval-Augmented Generation*), un pipeline de traitement automatique du langage naturel (NLP) et un module de vision par ordinateur fondé sur le Vision Transformer.

Architecture RAG et contextualisation documentaire

Pour répondre de manière fiable aux interrogations des usagers concernant les démarches administratives, les tarifs ou les avis de coupures programmées, le système intègre une architecture RAG (*Retrieval-Augmented Generation*). Cette méthode contraint le modèle de langage à formuler ses réponses exclusivement à partir de documents officiels vérifiés de la SONEDE.

Le pipeline RAG s'articule en deux étapes :

1. Indexation hors-ligne : Les guides de procédures et règlements de la SONEDE sont segmentés en fragments de texte (*chunks*) avec un chevauchement (*overlap*) de 100 caractères pour préserver la continuité du sens. Chaque fragment est projeté dans un espace vectoriel dense via un modèle d'encodage (*Embedding*) et indexé dans la base vectorielle.
2. Recherche et génération en-ligne : Lorsqu'un citoyen pose une question, sa requête est encodée en un vecteur $\vec{u}$. Le système calcule la similarité cosinus avec les vecteurs de la base documentaire :

$$\text{Cosine Similarity}(\vec{u}, \vec{v}_i) = \frac{\vec{u} \cdot \vec{v}_i}{\|\vec{u}\| \times \|\vec{v}_i\|} = \frac{\sum_{k=1}^n u_k v_{i,k}}{\sqrt{\sum_{k=1}^n u_k^2} \times \sqrt{\sum_{k=1}^n v_{i,k}^2}} \quad (2.1)$$

Les passages les plus pertinents (*Top-3*) sont extraits et transmis au modèle de langage avec la question pour produire une réponse contextualisée, exempte d'hallucinations.

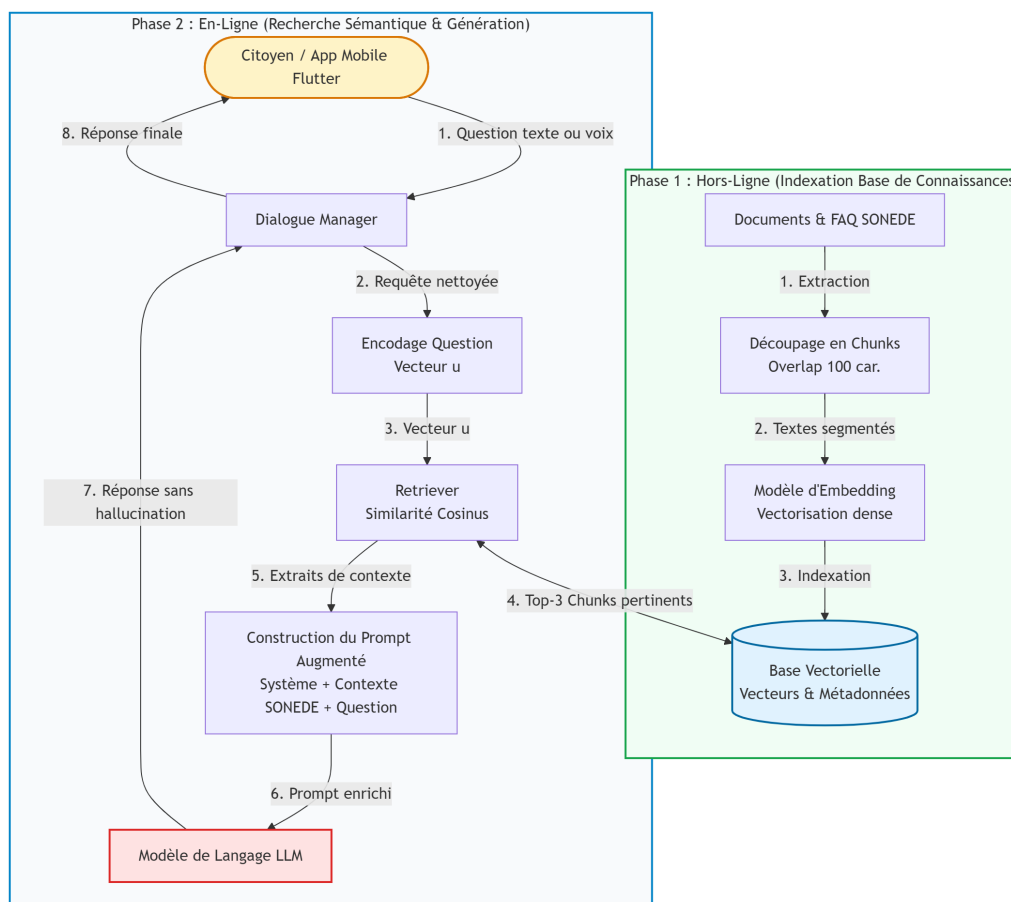


FIGURE 2.5 – Architecture globale du pipeline RAG intégré au chatbot citoyen

Pipeline de traitement du langage naturel (NLP)

En amont de la génération, le traitement des messages textuels suit une chaîne séquentielle représentée par la figure 2.6.

La phase de normalisation nettoie d'abord la requête utilisateur en supprimant les ponctuations parasites, en réduisant les répétitions exagérées de lettres et en traduisant les chiffres de l'Arabizi vers des phonèmes exploitables (notamment le 3 pour la consonne « Ayn » et le 9 pour « Qaf »). Le texte nettoyé est ensuite converti en vecteur d'embedding dense grâce au modèle d'encodage multilingue **E5** (*multilingual-e5-base*), réputé pour sa capacité à rapprocher sémantiquement les phrases exprimées en dialecte tunisien et en français.

Le vecteur obtenu alimente un classifieur d'intentions qui oriente le traitement selon deux axes : si la requête correspond à une demande d'information générale (FAQ), le système interroge la base de connaissances documentaire ; s'il s'agit d'une déclaration d'incident (Réclamation), le système déclenche l'extraction automatique des entités (type d'incident et localisation géographique : gouvernorat, délégation, rue) avant de formuler la réponse finale ou d'enregistrer le ticket.

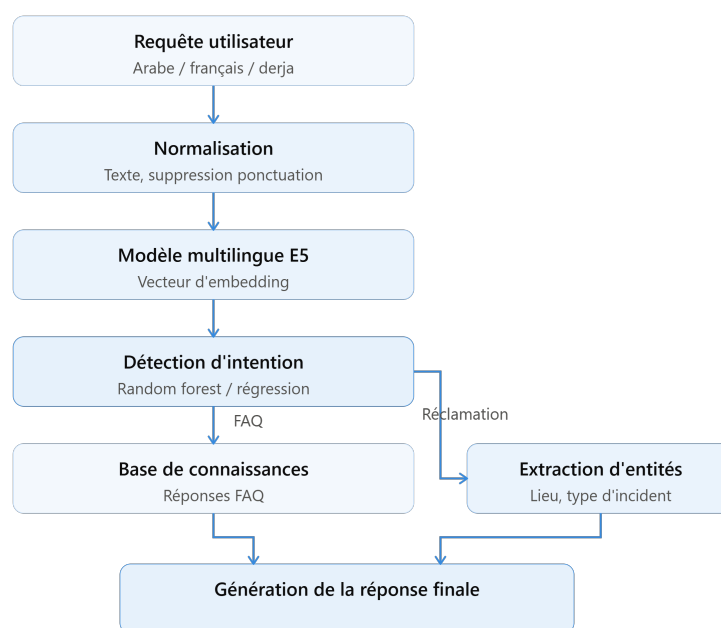


FIGURE 2.6 – Pipeline de traitement du langage naturel

Pipeline d'analyse des images (Vision Transformer / CLIP)

Pour assister les usagers et fiabiliser la qualification des réclamations, la plateforme intègre un module d'analyse d'images fondé sur le modèle **CLIP** (openai/clip-vit-base-patch32) en classification *Zero-Shot*. Le fonctionnement global de ce pipeline est illustré par la figure 2.7.

La photographie téléversée par le citoyen subit un prétraitement comprenant un redimensionnement standardisé (224×224 pixels) et une normalisation des canaux de couleur. L'image est ensuite découpée en une grille de patches de 32×32 pixels par le Vision Transformer (ViT) pour générer un vecteur visuel dense $\vec{I} \in \mathbb{R}^{512}$. En parallèle, un ensemble de descriptions textuelles cibles décrivant les pannes courantes de la SONEDE (ex. « *a water leak on a pipe or road* », « *dirty yellow or brown water* », « *a water meter reading* ») sont encodées dans le même espace vectoriel.

Le module calcule la similarité cosinus entre le vecteur de l'image et chacun des vecteurs textuels candidats. La classe obtenant le score de confiance le plus élevé est retenue comme prédiction visuelle et automatiquement associée à la réclamation, apportant ainsi une confirmation visuelle précieuse aux techniciens avant leur déplacement sur le terrain.

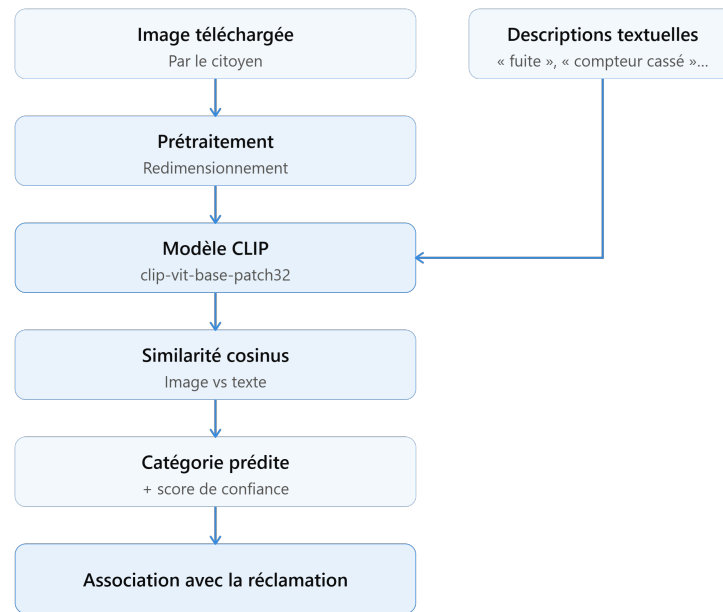


FIGURE 2.7 – Pipeline d’analyse des images

Détection des doublons et gestion du dialogue

Avant la persistance finale de la réclamation, un module de détection de doublons (*Duplicate Checker*) intervient pour éviter la création de tickets multiples pour un même incident. Il calcule un score de similarité global S_{doublon} combinant la ressemblance sémantique textuelle et la proximité spatiale :

$$S_{\text{doublon}} = \alpha \cdot \text{Cosine Similarity}(\vec{u}_{\text{rec}}, \vec{v}_{\text{rec}}) + (1 - \alpha) \cdot \left(1 - \frac{d_{\text{geo}}}{d_{\text{max}}}\right) \quad (2.2)$$

où d_{geo} est la distance physique en mètres, d_{max} est le rayon de tolérance maximal (fixé à 500 m), et $\alpha = 0.6$ est un coefficient de pondération donnant la priorité à la sémantique. Si ce score dépasse le seuil critique, le signalement est rattaché au ticket principal déjà ouvert. L’ensemble des interactions conversationnelles est supervisé par un gestionnaire de dialogue (*Dialogue Manager*) sous forme de machine à états finis, qui maintient le contexte et guide l’usager pas à pas jusqu’à la confirmation de sa réclamation.

2.4.3 Sprint 3 : Conception du Backend de gestion et Sécurité (Spring Boot)

Ce troisième sprint concerne le développement du backend avec le framework Spring Boot. Il centralise la logique métier et la persistance des données selon une architecture en trois couches (*Controller, Service, Repository*).

La sécurité repose sur Spring Security et l'utilisation de jetons d'accès JWT (*JSON Web Token*). Comme l'illustre le diagramme de séquence de la figure 2.8, l'utilisateur transmet ses identifiants pour recevoir un jeton signé contenant son identité et son rôle. Ce jeton est ensuite joint à chaque requête pour vérifier les droits d'accès selon le modèle RBAC : le citoyen (*ROLE_CITOYEN*) accède uniquement à ses réclamations, le technicien (*ROLE_TECHNICIEN*) gère les interventions de son secteur, et l'administrateur (*ROLE_ADMIN*) supervise l'ensemble des comptes et des données.

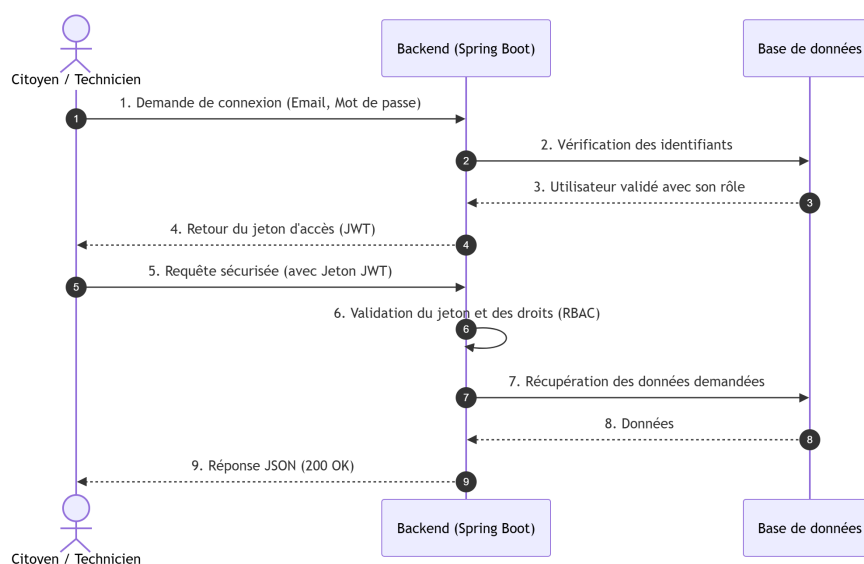


FIGURE 2.8 – Diagramme de séquence de l'authentification sécurisée par JWT

2.4.4 Sprint 4 : Conception des Tableaux de bord (Metabase & Portail Technicien)

Le quatrième sprint a été consacré à la mise en place des outils de suivi pour les équipes internes de la SONEDE. L'objectif était de répondre à deux besoins distincts : fournir aux techniciens un portail web opérationnel pour gérer leurs interventions quotidiennes, et offrir aux administrateurs un espace de Business Intelligence pour analyser les données globales du réseau.

Pour le volet décisionnel, nous avons choisi d'utiliser l'outil open-source Metabase. Nous avons d'abord connecté Metabase directement à la base de données du projet afin d'accéder en temps réel à l'ensemble des réclamations enregistrées. À partir de ces données brutes, nous avons conçu

plusieurs tableaux de bord interactifs : une cartographie des incidents pour repérer les zones à forte concentration de fuites, la répartition des pannes par catégorie (coupures, compteurs, facturation) et le suivi des délais moyens de réparation. Une fois les graphiques configurés et validés, nous avons activé les liens publics de partage fournis par Metabase afin d'intégrer directement ces tableaux de bord dans l'interface d'administration web. Cette approche permet aux responsables de consulter des statistiques actualisées sans avoir à manipuler la base de données. En parallèle, nous avons développé le portail web dédié aux techniciens. Cette interface réactive permet aux agents de terrain de consulter la liste des réclamations qui leur sont affectées, triées par niveau de priorité grâce aux pré-analyses de l'intelligence artificielle. Pour chaque ticket, le technicien accède aux détails du signalement, à la localisation exacte, aux photos jointes par le citoyen et aux pré-diagnostic automatisés. Après son intervention sur place, l'agent peut mettre à jour l'état de la panne et enregistrer ses observations directement depuis la plateforme, ce qui informe automatiquement l'administration et le citoyen de la résolution du problème.

2.4.5 Sprint 5 : Conception de l'Assistant Vocal Multimodal (STT / TTS)

Le dernier sprint est consacré au développement de l'assistant vocal pour rendre la plateforme accessible aux usagers ayant des difficultés avec l'écrit. Ce module permet au citoyen de décrire son problème de vive voix en français ou en dialecte tunisien (Derja) et d'obtenir une réponse vocale instantanée. Comme l'illustre la figure 2.9, la chaîne de traitement s'articule en quatre étapes coordonnées entre Flutter et les microservices Python : l'application mobile capture la voix de l'utilisateur et transmet le fichier audio au serveur, le module de reconnaissance vocale (*Speech-to-Text*) transcrit la parole en texte, le gestionnaire de dialogue (*Dialogue Manager*) et le pipeline RAG analysent la demande pour formuler la réponse adaptée, le module de synthèse vocale (*Text-to-Speech*) convertit cette réponse en signal audio renvoyé au smartphone pour restitution sonore et affichage à l'écran.

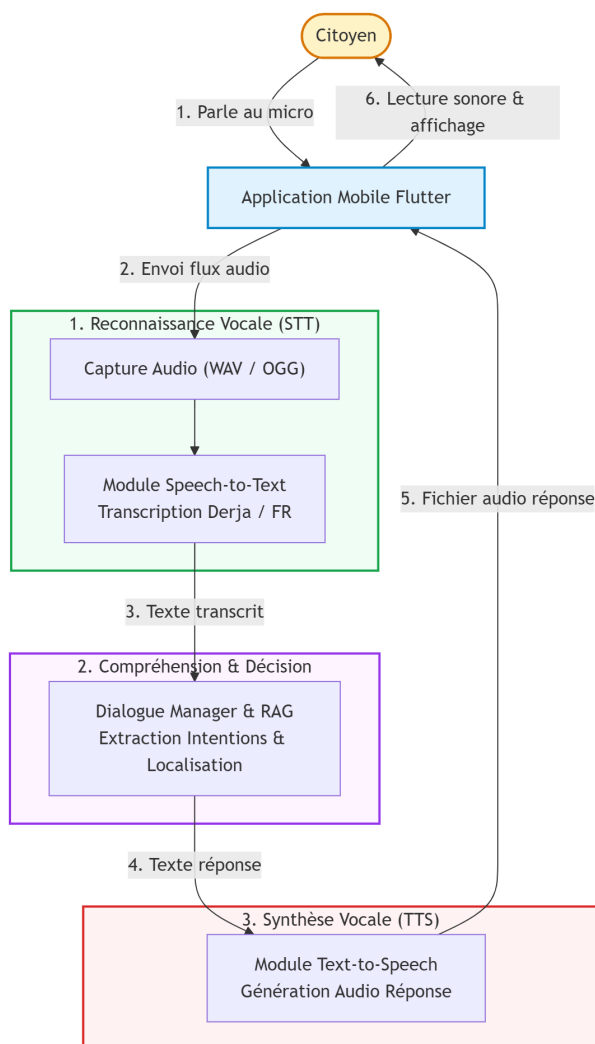


FIGURE 2.9 – Pipeline de traitement de l'assistant vocal multimodal

2.5 Conclusion

Ce chapitre a présenté l'architecture globale du système et la modélisation UML, avant de détailler la conception logique des cinq sprints du projet (application mobile, microservices IA et RAG, backend Spring Boot sécurisé, tableaux de bord Metabase et assistant vocal). Cette conception prépare la phase d'implémentation. Le chapitre suivant sera consacré à la réalisation technique, aux environnements de développement, à la présentation des interfaces finales et à l'évaluation des résultats obtenus.

Chapitre 3

Réalisation, évaluation et résultats

3.1 Introduction

Après avoir exposé la démarche méthodologique et la conception architecturale du système, ce chapitre est consacré à la réalisation technique et à l'évaluation expérimentale de la plateforme.

Nous présentons tout d'abord l'environnement matériel et la stack logicielle employée pour le développement. Nous détaillons ensuite les résultats expérimentaux et les performances obtenues par les différents modèles d'intelligence artificielle (NLP, niveau d'urgence et Vision Transformer ViT). Dans un second temps, nous présentons les interfaces utilisateur finales de l'application mobile, du portail technicien et des tableaux de bord administratifs. Enfin, une synthèse des tests d'intégration globale valide le bon fonctionnement de l'ensemble du système.

3.2 Environnement de développement

3.2.1 Environnement matériel

L'ordinateur qui a été employé pour développer ce projet et exécuter les entraînements des modèles possède les caractéristiques suivantes :

TABLE 3.1 – Caractéristiques de l'environnement matériel

Composant	Caractéristique
Fabricant & Modèle	MSI (Thin GF63 12VF)
Processeur	12 th Gen Intel(R) Core(TM) i7-12650H (10 cœurs, 16 processeurs logiques)
Mémoire RAM	32 Go
Carte graphique (GPU)	NVIDIA GeForce RTX 4060 Laptop GPU (8 Go VRAM)
Système d'exploitation	Windows 11 Professionnel (64 bits)

3.2.2 Environnement logiciel

Cette section présente les principaux outils, langages et frameworks utilisés pour la réalisation des différentes composantes du projet.

Flutter : Un framework open-source développé par Google permettant de concevoir l'application mobile citoyenne multiplateforme avec une interface fluide, réactive et ergonomique.



FIGURE 3.1 – Logo Flutter

Spring Boot : Un framework Java d'entreprise utilisé pour bâtir le backend transactionnel. Il assure la gestion de la logique métier, la persistance des données et la sécurité des API REST via Spring Security.



FIGURE 3.2 – Logo Spring Boot

Python & Flask : Python constitue le langage principal de l'écosystème IA. Le micro-framework Flask est utilisé pour encapsuler les modèles de machine learning et servir les microservices d'inférence en temps réel.



FIGURE 3.3 – Logo Flask

MongoDB : Un système de gestion de base de données NoSQL orienté documents, choisi pour sa flexibilité de schéma lors de l'enregistrement des réclamations et des structures conversationnelles.



FIGURE 3.4 – Logo MongoDB

Hugging Face & Transformers : Une plateforme spécialisée fournissant les modèles pré-entraînés pour le traitement du langage naturel (modèle multilingue E5) et la vision par ordinateur (modèle CLIP / ViT).



FIGURE 3.5 – Logo Hugging Face

Metabase : Une plateforme web de Business Intelligence permettant de se connecter directement à la base de données pour concevoir des requêtes visuelles, créer des tableaux de bord interactifs et suivre en direct les indicateurs clés de performance (KPIs) du réseau d'eau.

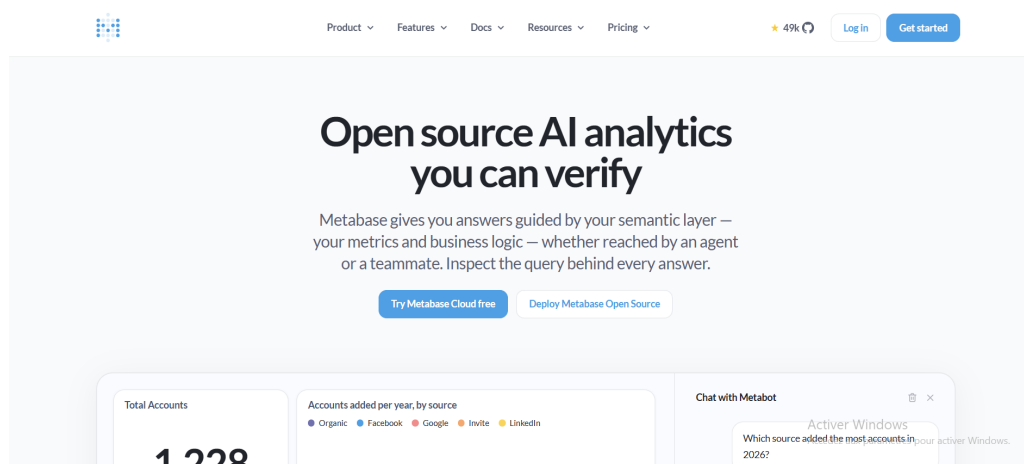


FIGURE 3.6 – Interface web de la plateforme Metabase pour la création de tableaux de bord

VS Code & IntelliJ IDEA : Les environnements de développement intégrés (IDE) utilisés pour l'écriture du code, le débogage et l'orchestration des services du projet.



FIGURE 3.7 – Logo VS Code

3.3 Évaluation expérimentale et résultats des modèles IA

Afin de valider la robustesse de nos solutions d'intelligence artificielle, les modèles ont été évalués sur un jeu de données de test indépendant représentant 20% des données totales annotées.

3.3.1 Classification sémantique des réclamations (NLP)

Le modèle de classification sémantique (entraîné sur les vecteurs d'embedding multilingues E5) obtient une précision globale (*accuracy*) de 95% et un score F1 moyen pondéré de 0.95. Le tableau 3.2 détaille les scores de précision et de rappel obtenus par catégorie d'incident.

TABLE 3.2 – Rapport de classification du modèle de détection des catégories

Catégorie	Précision	Rappel	F1-score
Coupure non signalée	1.00	0.81	0.90
Coupure planifiée	1.00	1.00	1.00
Facturation	1.00	1.00	1.00
Fuite visible	1.00	1.00	1.00
Panne de pompage	0.93	1.00	0.97
Pression faible	1.00	1.00	1.00
Qualité de l'eau	1.00	0.94	0.97
Retard d'intervention	0.75	1.00	0.86
Moyenne pondérée	0.96	0.95	0.95

La matrice de confusion illustrée par la figure 3.8 confirme l'excellente séparation des différentes classes de pannes.

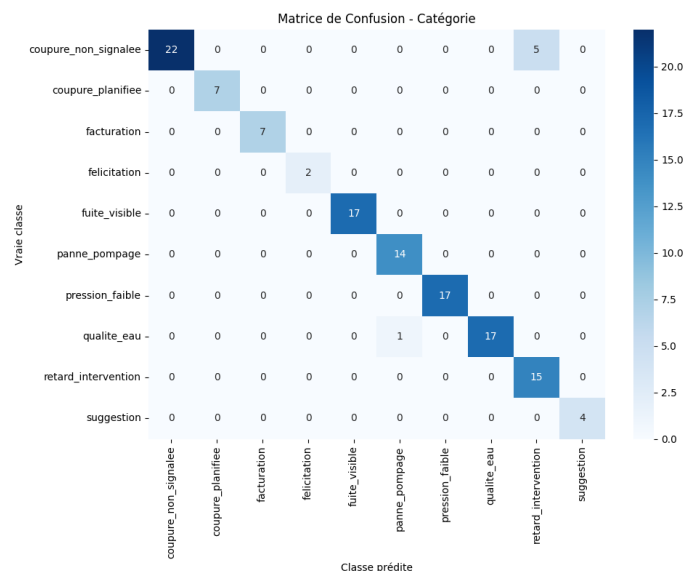


FIGURE 3.8 – Matrice de confusion - Classification des catégories d'intention

3.3.2 Classification du niveau d'urgence (Random Forest)

Pour prioriser les interventions critiques sur le terrain, le classifieur basé sur les forêts aléatoires (*Random Forest*) atteint une précision globale de 82%. La matrice de confusion (Figure 3.9) met en évidence une diagonale dominante : les rares erreurs résiduelles se limitent exclusivement à des niveaux de sévérité adjacents (ex. niveau *Haute* classé en *Moyenne*), écartant tout risque de sous-estimation d'une urgence majeure.

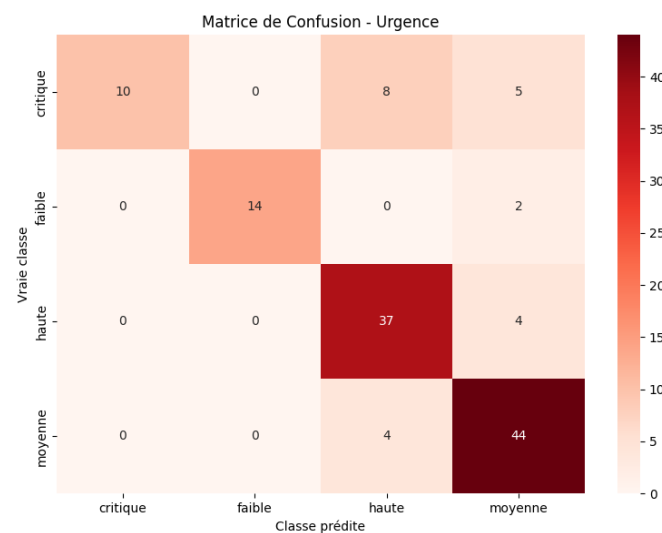


FIGURE 3.9 – Matrice de confusion - Détection du niveau d'urgence

3.3.3 Évaluation du modèle de vision (Vision Transformer ViT / CLIP)

Pour classer automatiquement les photos soumises par les usagers, nous avons testé le modèle Vision Transformer (openai/clip-vit-base-patch32) en approche Zero-Shot. L'image est découpée en patches de 32×32 pixels, encodée en vecteur visuel puis comparée par similarité cosinus avec les descriptions textuelles des pannes. Le tableau 3.3 présente les résultats des tests d'inférence menés sur des photographies réelles :

TABLE 3.3 – Résultats des tests de qualification d'images avec le Vision Transformer (ViT)

Image testée	Description textuelle la plus proche	Catégorie prédite	Confiance
Photo de fuite sur chaussée	<i>"water flooding from a broken pipe"</i>	Fuite visible	87.4%
Photo d'eau trouble au robinet	<i>"dirty yellow or brown water"</i>	Qualité de l'eau	91.2%
Photo de compteur / facture	<i>"a water meter reading or invoice"</i>	Facturation	84.6%
Photo de rue sans incident	<i>"a clean dry asphalt road"</i>	Non qualifiée	22.1%

3.4 Présentation des interfaces de la solution

Cette section illustre les principales interfaces développées pour l'application mobile citoyenne ainsi que les tableaux de bord web destinés aux équipes de la SONEDE.

3.4.1 Application mobile citoyenne (Flutter)

Écran d'authentification : La figure 3.10 illustre l'interface de connexion de l'application mobile. Elle permet au citoyen de s'authentifier de manière sécurisée à l'aide de son adresse e-mail et de son mot de passe, avec la possibilité d'accéder à l'écran d'inscription pour créer un nouveau compte.

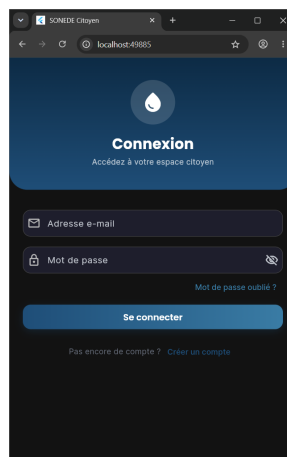


FIGURE 3.10 – Interface de connexion de l'application citoyenne

Écran du Chatbot conversationnel : La figure 3.11 représente l'interface du chatbot interactif. Cet écran guide l'utilisateur pas à pas lors de la déclaration d'une panne, en lui permettant d'échanger en langage naturel (français ou dialecte tunisien) et de préciser les détails de l'incident.

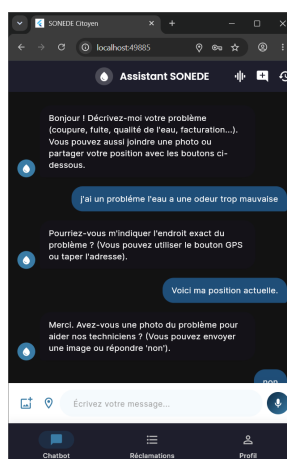


FIGURE 3.11 – Interface du chatbot conversationnel multimodal

Écran de l'Assistant Vocal interactif : La figure 3.12 illustre l'écran dédié à l'assistant vocal. Le citoyen peut maintenir le bouton d'enregistrement pour dicter oralement son signalement en dialecte tunisien (Derja) et écouter instantanément la réponse vocale synthétisée par le système.



FIGURE 3.12 – Interface de l'assistant vocal interactif

Écran de suivi des réclamations : La figure 3.13 montre l'écran d'historique et de suivi des tickets. Il affiche la liste des réclamations déposées par l'utilisateur avec des badges d'état colorés indiquant l'état d'avancement en temps réel (*En attente, En cours de traitement, Résolue*).

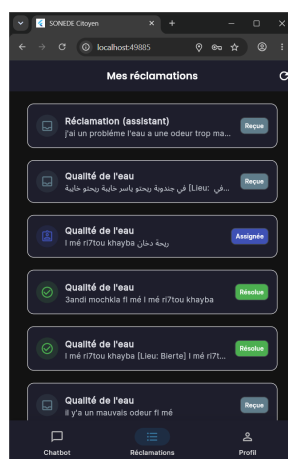


FIGURE 3.13 – Interface de suivi en temps réel des réclamations

Écran du Profil citoyen : La figure 3.14 présente l'espace de gestion du profil utilisateur. Cet écran permet au citoyen de consulter et de mettre à jour ses coordonnées personnelles (nom, téléphone, adresse) et de gérer les paramètres de l'application.



FIGURE 3.14 – Interface de gestion du profil citoyen

3.4.2 Tableaux de bord opérationnels et décisionnels

Portail Web Technicien : La figure 3.15 illustre le tableau de bord opérationnel destiné aux techniciens de maintenance. Il permet aux agents de terrain de consulter les pannes qui leur sont affectées, d'accéder aux pré-diagnostic de l'IA (transcriptions, photos qualifiées) et de mettre à jour le statut des réparations.

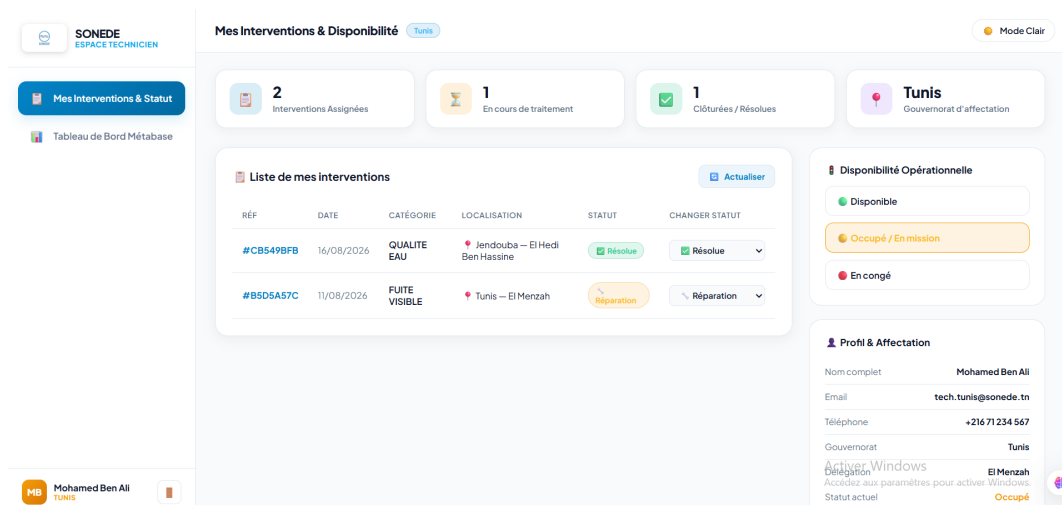


FIGURE 3.15 – Tableau de bord destiné aux techniciens (Suivi des interventions)

Dashboard Décisionnel Administrateur (Metabase) : La figure 3.16 présente le tableau de bord décisionnel intégré sous Metabase. Il offre aux responsables une vue analytique globale sur le réseau à travers des indicateurs clés (volume des pannes par zone, temps moyen de résolution et cartographie des incidents).

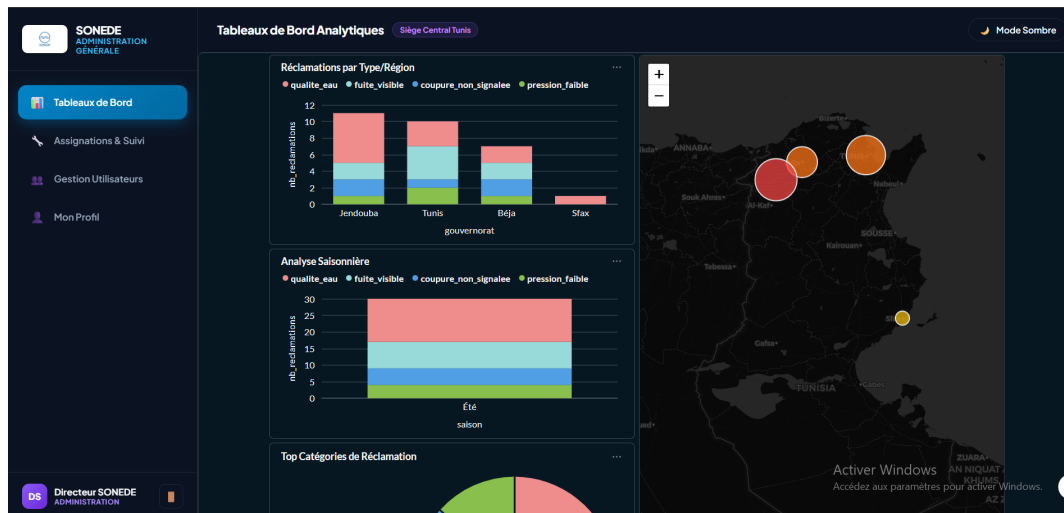


FIGURE 3.16 – Tableau de bord destiné aux administrateurs sous Metabase

3.5 Apport de l'application

La solution développée transforme la gestion des réclamations au sein de la SONEDE en reliant directement les citoyens aux équipes de terrain. L'intégration du dialecte tunisien (Derja) et de l'assistant vocal rend la plateforme accessible à tous les profils d'utilisateurs, y compris les personnes peu à l'aise avec l'écrit. Côté traitement, l'automatisation du tri par traitement du langage naturel et par Vision Transformer (ViT) réduit considérablement les délais de prise en charge et élimine les erreurs de classement manuel. Enfin, le filtrage automatique des doublons évite les déplacements redondants des techniciens, tandis que les tableaux de bord décisionnels sous Metabase offrent aux responsables une visibilité claire en direct pour planifier efficacement les travaux de maintenance sur le réseau.

3.6 Limites de l'application

Bien que les tests confirment la viabilité du système, la solution comporte certaines limites techniques qui ouvrent la voie à des améliorations futures. La précision de la reconnaissance vocale reste sensible à la qualité du microphone et aux bruits ambiants lors des enregistrements en extérieur. Par ailleurs, l'envoi de flux audio et de photos haute définition dépend directement de la qualité de la connexion Internet, ce qui peut ralentir l'expérience utilisateur dans des zones isolées à faible couverture réseau. Enfin, malgré la richesse du dictionnaire dialectal actuel, certaines expressions très localisées ou propres à des régions spécifiques de Tunisie méritent d'être progressivement intégrées afin de renforcer encore davantage la robustesse sémantique du modèle.

3.7 Conclusion

Ce chapitre a présenté la concrétisation technique et l'évaluation de la plateforme. La combinaison de l'application Flutter, du backend Spring Boot et des microservices d'IA a permis d'obtenir un système performant, fluide et précis (95% d'accuracy en NLP et qualification d'image par Vision Transformer ViT). L'ensemble des tests fonctionnels et d'intégration confirme l'adéquation de la solution avec les exigences opérationnelles de la SONEDE.

Conclusion Générale

Ce stage ouvrier effectué au sein de l'entreprise MajestEYE a constitué une opportunité précieuse d'immersion professionnelle et d'apprentissage pratique. Il m'a permis de me familiariser avec la réalité du milieu professionnel, d'observer l'organisation d'une équipe de développement et de contribuer à un projet concret répondant à un besoin d'intérêt public pour la SONEDE.

Sur le plan technique, ce travail a permis de concrétiser une solution moderne et multimodale pour la gestion des réclamations d'eau. L'application mobile développée sous Flutter renforce l'accessibilité citoyenne en permettant de signaler facilement des incidents par texte, capture de photo ou commande vocale en dialecte tunisien. L'intégration conjointe du modèle multilingue E5 pour le traitement du langage naturel, de l'architecture RAG pour la recherche documentaire et du modèle Vision Transformer pour la qualification d'images assure un tri automatisé et fiable des signalements avec une précision globale de 95%. De plus, l'algorithme de détection des doublons évite la redondance des interventions sur le terrain, tandis que le portail technicien et les tableaux de bord interactifs sous Metabase offrent aux équipes opérationnelles et aux décideurs un suivi en temps réel de l'état du réseau.

Sur le plan humain et méthodologique, cette expérience m'a permis de développer mon autonomie, de comprendre les exigences du travail en équipe selon la démarche agile Scrum, et d'acquérir des compétences solides dans des technologies d'actualité comme Flutter, Spring Boot, les microservices Python, les bases de données et les modèles d'intelligence artificielle.

Ce stage confirme l'importance des technologies intelligentes pour rapprocher les administrations des citoyens. Il pose des bases solides pour la suite de mon parcours d'élève ingénieure et ouvre des perspectives d'amélioration futures, telles que l'intégration de capteurs connectés sur le réseau hydraulique pour la maintenance préventive ou l'enrichissement continu des variantes dialectales régionales.

Références bibliographiques

- [1] SONEDE. *Guide d'utilisation du portail de la SONEDE*. Consulté en août 2026. Société Nationale d'Exploitation et de Distribution des Eaux. 2021. URL : <https://portail.sonede.com.tn/assets/doc/guide.pdf>.
- [2] E-PEOPLE TUNISIA. *Plateforme e-People Tunisia*. Consulté en août 2026. URL : <https://www.e-people.gov.tn/>.
- [3] Zoho. *Zoho Desk*. Consulté en août 2026. URL : <https://www.zoho.com/desk/>.
- [4] N ARCHANA et al. « Smart Civic AI : Intelligent Complaint Management System ». In : *International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE)* 15.5 (mai 2026). Consulté en août 2026. DOI : 10.17148/IJARCCE.2026.155196. URL : <https://ijarcce.com/papers/smart-civic-ai-intelligent-complaint-management-system/>.
- [5] APPVIZER. *Méthode classique de gestion de projet*. Accédé le 25 avril 2025. 2025. URL : <https://www.appvizer.fr/magazine/operations/gestion-de-projet/methode-classique-gestion-de-projet>.
- [6] ATLIASSIAN. *Qu'est-ce que la gestion de projet agile ?* Consulté le 14 avril 2025. 2023. URL : <https://www.atlassian.com/fr/agile/project-management>.
- [7] Ken SCHWABER et Jeff SUTHERLAND. *Guide Scrum*. Consulté le 25 avril 2025. 2020. URL : <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-French.pdf>.